

Simulations for Input/Output Subsystems

Top Shelf Mover

This will be a breakdown of the simulation processed that I used to create the functions of the input/output section. I used the same implementations that were described in my Tradeoff analysis with a few adaptations to account for parts selection and improvements to implementations.

Part 1 – Startup and Pushbutton

I used Tinker CAD to create my simulation. I started off by creating my pushbutton implementation for the subsystem. The pushbutton is the main control unit, with the pushbutton controlling the motors that are connected to a belt that will move the shelves of our project. I started out basic with a pushbutton and an RGB LED light. I connected everything through an Arduino and a breadboard.

Pseudo Code:

Define the pins for the Arduino connections

Create the variables for the pins (int)

Create a variable for the push button (int)

Within void setup, create label the inputs and outputs of the pins

Within void loop, create a read from the push button, and then use an if else statement to have the RGB be the correct color depending on the state of the push button

The simulation is presented below:

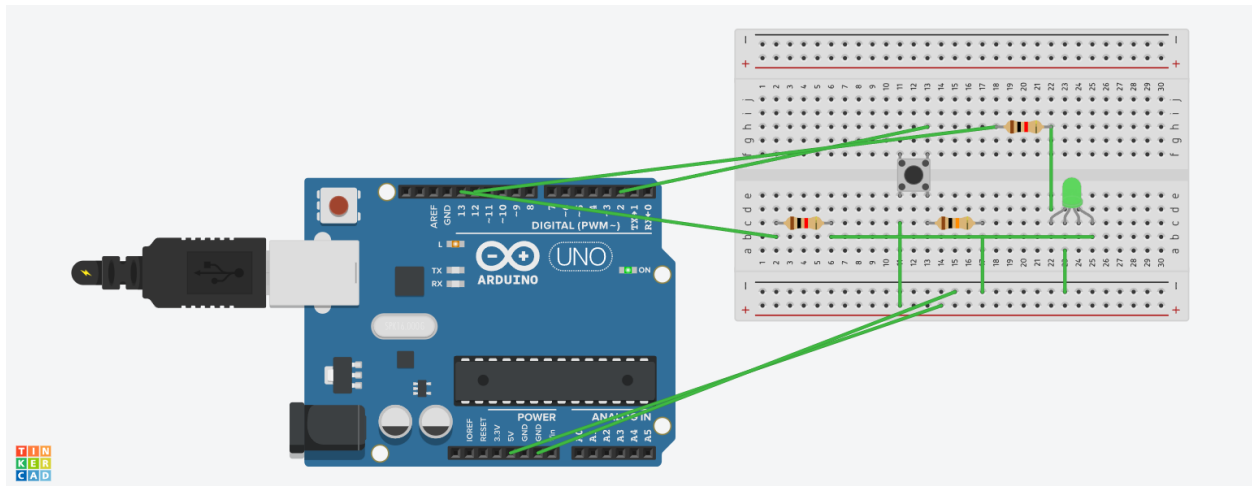


Figure 1: Pushbutton On

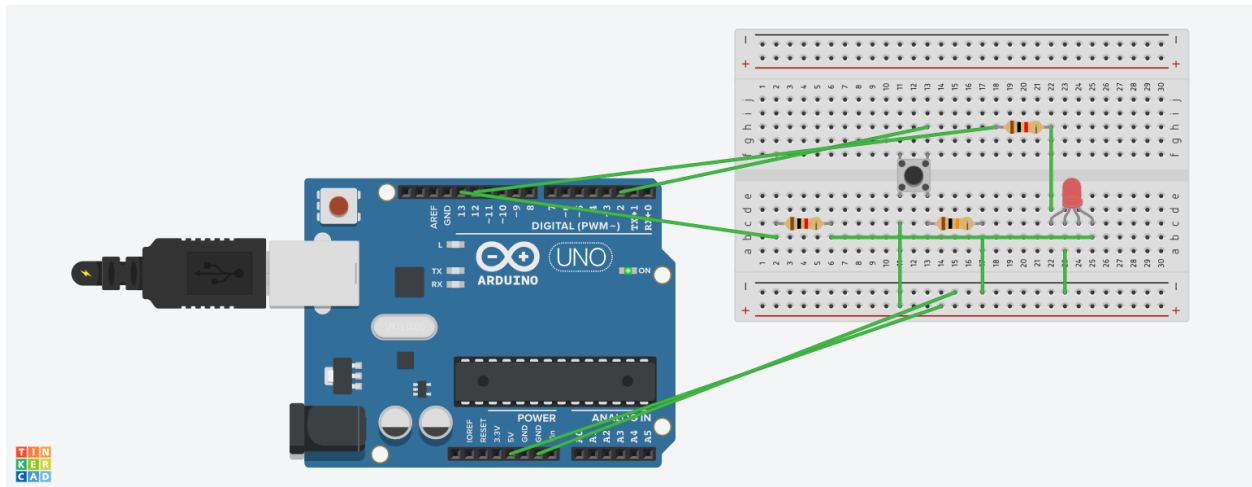


Figure 2: Pushbutton Off

As you can see from the above figures, when the pushbutton is pressed, the RGB LED is green. This means that the operation is going as planned once the pushbutton is pressed. The RGB LED is red by default, which signals a ready state/stopped state. For future implementations, the RGB LED will show red when the ultrasonic sensor detects an object, even if the button is pressed.

Code:

```

1  const int buttonPin = 2;  // the number of the pushbutton pin
2  const int GPin = 13;      // the number of the Gpin
3  const int RPin = 12;
4
5  // variables will change:
6  int buttonState = 0;  // variable for reading the pushbutton status
7
8  void setup() {
9      // initialize the Green pin as an output:
10     pinMode(GPin, OUTPUT);
11     // initialize the pushbutton pin as an input:
12     pinMode(buttonPin, INPUT);
13     pinMode(RPin, OUTPUT);
14 }
15
16 void loop() {
17     // read the state of the pushbutton value:
18     buttonState = digitalRead(buttonPin);
19
20     // check if the pushbutton is pressed. If it is, the buttonState
21     if (buttonState == HIGH) {
22         // turn RGB on:
23         digitalWrite(GPin, HIGH);
24         digitalWrite(RPin, LOW);
25     } else {
26         // turn RED PIN on:
27         digitalWrite(RPin, HIGH);
28         digitalWrite(GPin, LOW);
29     }
30 }
31

```

Part 2 – Ultrasonic Sensor Added

With the basics established within the implementation for the pushbutton along with the RGB LED, I implemented the ultrasonic sensor to work along with the pushbutton. The purpose of our ultrasonic sensor is for safety due to the moving shelves, so if an object is deemed too close, the operation will not run. I added an additional if statement within the if else statement of the pushbutton to negate the operation if an issue were to occur.

Pseudo Code:

Define pins for sensor

Add variables for distance

Within void setup, assign the inputs and outputs

Within void loop, add an if statement to overtake normal operation if prompted

Ultrasonic Sensor simulation:

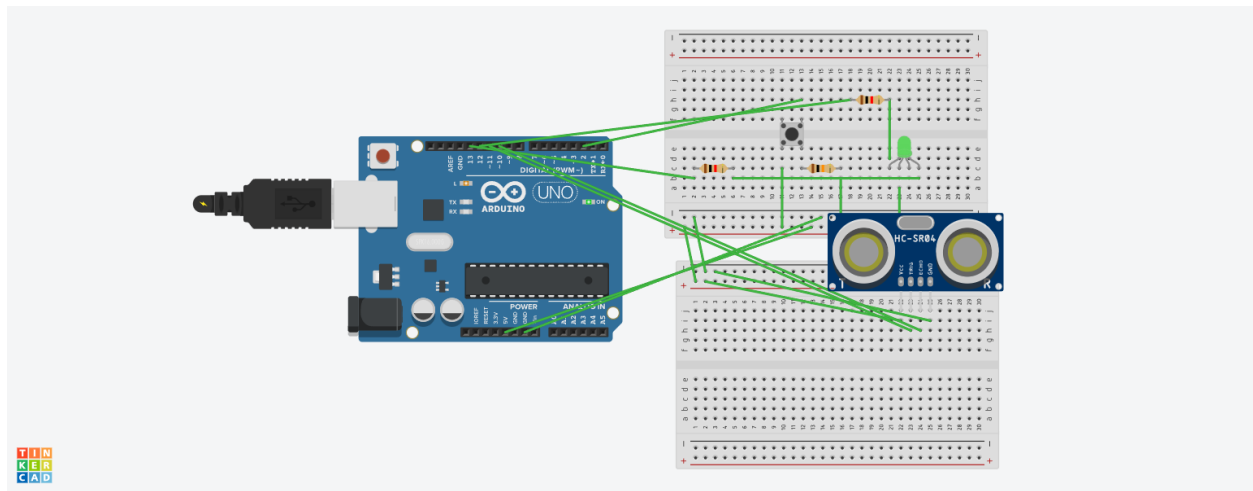


Figure 3: Ultrasonic Sensor (object far)

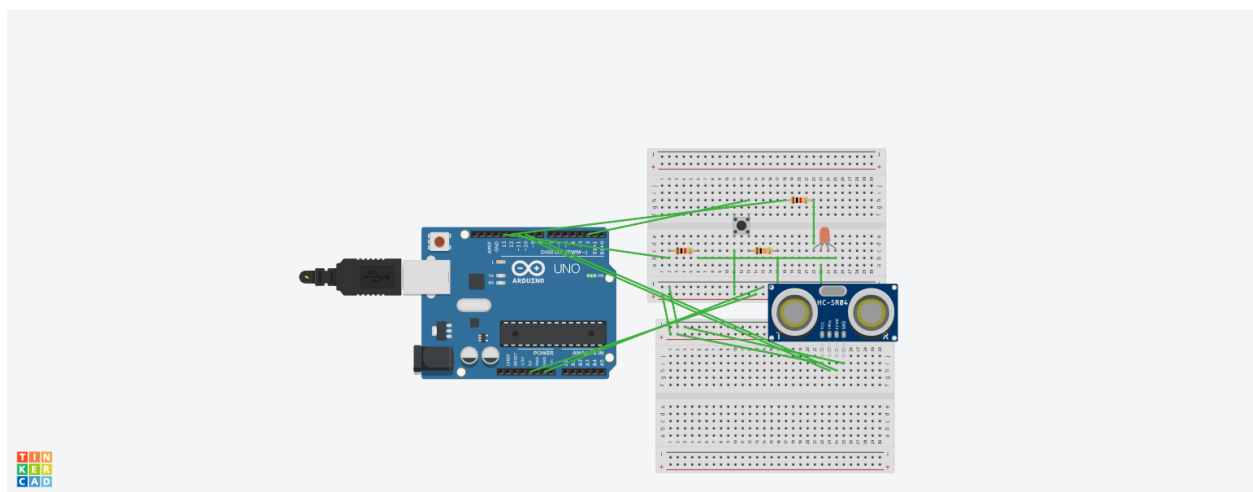


Figure 4: Ultrasonic Sensor (object close)

Figure 3 shows that the operation continues as normal even with the ultrasonic sensor implemented. However, once an object is within a certain distance, the RGB LED flips to red. In the motor implementation, I will provide a figure that shows a figure within a close distance that overrides the normal operations.

Code:

```
const int buttonPin = 2; // the number of the pushbutton pin
const int GPin = 13;     // the number of the Gpin
const int RPin = 12;
const int TRIG_PIN = 11; // Arduino pin connected to Ultrasonic sensor
const int ECHO_PIN = 10; // Arduino pin connected to Ultrasonic sensor
const int DISTANCE_THRESHOLD = 50; // centimeters

// variables will change:
int buttonState = 0; // variable for reading the pushbutton state
float duration_us, distance_cm;

void setup() {
  Serial.begin(9600);
  pinMode(TRIG_PIN, OUTPUT); // set arduino pin to output mode
  pinMode(ECHO_PIN, INPUT);  // set arduino pin to input mode
  // initialize the Green pin as an output:
  pinMode(GPin, OUTPUT);
  // initialize the pushbutton pin as an input:
  pinMode(buttonPin, INPUT);
  pinMode(RPin, OUTPUT);
}

void loop() {
  // generate 10-microsecond pulse to TRIG pin
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
```

```

31 duration_us = pulseIn(ECHO_PIN, HIGH);
32 // calculate the distance
33 distance_cm = 0.017 * duration_us;
34
35
36 buttonState = digitalRead(buttonPin);
37
38 // check if the pushbutton is pressed. If it is, the buttonStat
39 if (buttonState == HIGH) {
40     // turn RGB on:
41     digitalWrite(GPin, HIGH);
42     digitalWrite(RPin, LOW);
43
44     if(distance_cm < DISTANCE_THRESHOLD) {
45
46         digitalWrite(RPin, HIGH);
47         digitalWrite(GPin, LOW); // turn on LED
48     } else {
49         digitalWrite(RPin, LOW);
50         digitalWrite(GPin, HIGH);
51     }
52
53
54 // read the state of the pushbutton value:
55 } else {
56     // turn RED PIN on:
57     digitalWrite(RPin, HIGH);
58     digitalWrite(GPin, LOW);
59 }
60 }
61

```

Part 3: Motor Implementation

In the tradeoff analysis, I used another board (Motor Shield Rev 3) for motor control along with the Arduino. However, after creating our parts list, we decided not to implement the motor control with this method. I researched another implementation and found an alternative that works great with our pushbutton implementation. I just added the motor functions within the previously implemented if else statements.

Pseudo Code:

Define the pin for the motor function

Within void loop, add the motor function within the if else statements

Motor Implementation:

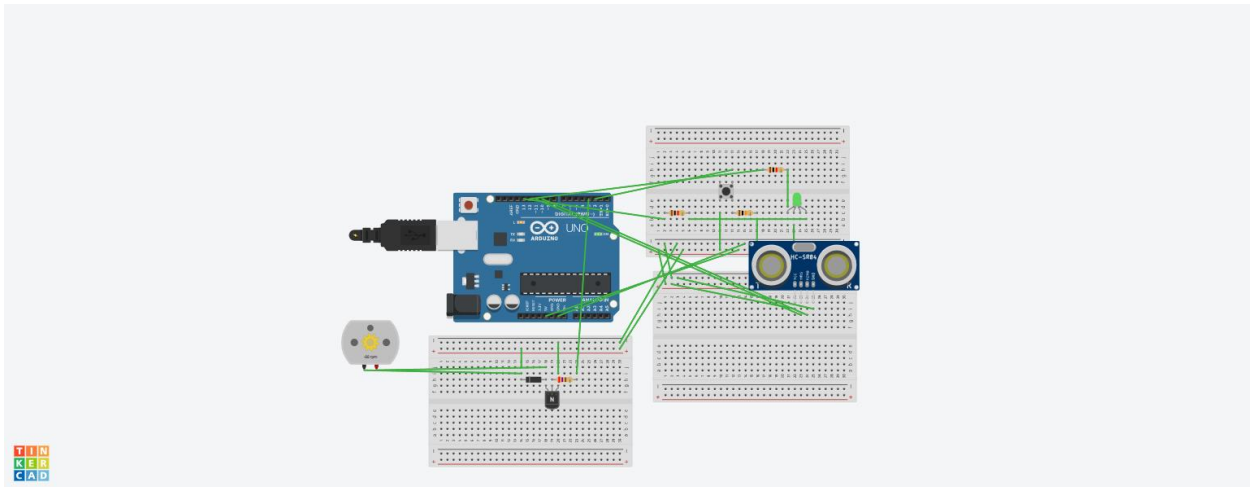


Figure 5: Motor On

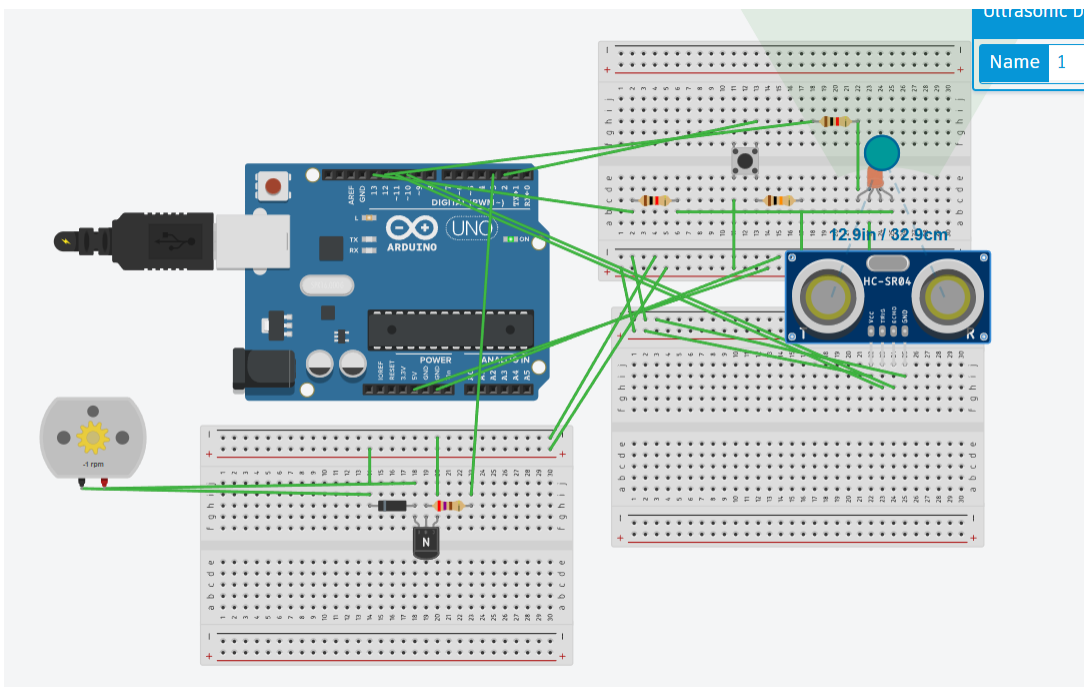


Figure 6: Ultrasonic Takeover

In Figure 5, the implementation shows the complete operation within the input output system, with the pushbutton getting pressed, the dc motor spinning, the Green LED showing

normal operation. Figure 6 shows that when an object is too close to the sensor (less than 50 cm), it shuts down the normal operation, and the motor is slowed to near 0 and the LED is changed to red.

Code:

```
1  const int buttonPin = 2; // the number of the pushbutton pin
2  const int GPin = 13;     // the number of the Gpin
3  const int RPin = 12;
4  const int TRIG_PIN = 11; // Arduino pin connected to Ultrasonic s
5  const int ECHO_PIN = 10; // Arduino pin connected to Ultrasonic s
6  const int DISTANCE_THRESHOLD = 50; // centimeters
7  const int motorPin = 3;
8
9  // variables will change:
10 int buttonState = 0; // variable for reading the pushbutton stat
11 float duration_us, distance_cm;
12
13
14 void setup() {
15   Serial.begin (9600);
16   pinMode(TRIG_PIN, OUTPUT); // set arduino pin to output mode
17   pinMode(ECHO_PIN, INPUT);  // set arduino pin to input mode
18   // initialize the Green pin as an output:
19   pinMode(GPin, OUTPUT);
20   // initialize the pushbutton pin as an input:
21   pinMode(buttonPin, INPUT);
22   pinMode(RPin, OUTPUT);
23 }
24
25 void loop() {
26   // generate 10-microsecond pulse to TRIG pin
27   digitalWrite(TRIG_PIN, HIGH);
28   delayMicroseconds(10);
29   digitalWrite(TRIG_PIN, LOW);
30
31   // measure duration of pulse from ECHO pin
```



```

32 duration_us = pulseIn(ECHO_PIN, HIGH);
33 // calculate the distance
34 distance_cm = 0.017 * duration_us;
35
36
37 buttonState = digitalRead(buttonPin);
38
39 // check if the pushbutton is pressed. If it is, the buttonStat
40 if (buttonState == HIGH) {
41     // turn RGB on:
42     digitalWrite(GPin, HIGH);
43     digitalWrite(RPin, LOW);
44     digitalWrite(motorPin, HIGH);
45
46     if (distance_cm < DISTANCE_THRESHOLD) {
47
48         digitalWrite(RPin, HIGH);
49         digitalWrite(GPin, LOW); // turn on LED
50         digitalWrite(motorPin, LOW);
51     }
52
53
54     // read the state of the pushbutton value:
55 } else {
56     // turn RED PIN on:
57     digitalWrite(RPin, HIGH);
58     digitalWrite(GPin, LOW);
59     digitalWrite(motorPin, LOW);
60 }
61 }
62

```

This is a good starting point for the initial implementation of the project. The final version will have more ultrasonic sensors, an ir breakbeam sensor, and another dc motor. I'm confident that our team can implement this design.

Implementation Sources:

RGB LED schematic and code - <https://projecthub.arduino.cc/semsemharaz/interfacing-rgb-led-with-arduino-b59902>

Ultrasonic sensor schematic and code - <https://arduinogetstarted.com/tutorials/arduino-ultrasonic-sensor-led>

DC motor - https://www.tutorialspoint.com/arduino/arduino_dc_motor.htm

